InfoBridge: A Retrieval-Augmented Generation System for Multi-Service Indian Government Document Querying

SMAI Assignment 3 — T10.7: Multi-Service Government RAG Chatbot

Abhay Sharma Ankit Kumar Indian Institute of Technology Hyderabad (IITH)
ai23btech11002@iith.ac.in
Hritik Ranjan
Indian Institute of Technology Hyderabad (IITH)
ai24btech11014@iith.ac.in

Abstract

We present **InfoBridge**, a production-grade Retrieval-Augmented Generation (RAG) chatbot that democratises citizen access to Indian government services. The system ingests official PDF documents spanning five service domains—Passport, Voter ID/EPIC, Driving Licence & RC, Income Tax, and Ayushman Bharat (PM-JAY)—and serves grounded, source-cited answers through a bilingual Streamlit interface. InfoBridge combines a PyMuPDF-based PDF extraction pipeline, a `sentence-transformers/all-MiniLM-L6-v2` embedding model (384-d), a FAISS flat inner-product index containing 21,020 vectors indexed from 3,812 document pages, and a Groq-hosted LLaMA-3 language model. A query normalisation layer translates Hinglish and Devanagari queries to English before retrieval; a lightweight conversational classifier routes greetings and portal-link requests directly to the LLM, bypassing retrieval. Qualitative evaluation over 42 queries spanning seven categories demonstrates 90% factual grounding, 100% citation accuracy, and 96% language-switching fidelity.

Keywords: Retrieval-Augmented Generation, Government Services, FAISS, Sentence Transformers, LLaMA-3, Multilingual NLP, Streamlit.

1 Introduction

India's digital governance ecosystem has expanded rapidly with platforms such as DigiLocker, UMANG, and service-specific portals. Despite this progress, a significant digital literacy gap persists: citizens often struggle to locate precise procedural information scattered across hundreds of official PDF manuals and government circulars [8].

Large Language Models (LLMs) offer a natural interface for querying unstructured corpora, but hallucinate on domain-specific questions without grounding [1]. Retrieval-Augmented Generation (RAG) [1] addresses this by conditioning generation on retrieved document passages, combining neural fluency with factual precision.

InfoBridge applies RAG specifically to five Indian government service categories that collectively affect hundreds of millions of citizens: passport issuance and renewal, voter registration and EPIC card management, driving licence and vehicle registration, income-tax filing, and Ayushman Bharat health coverage. The system is designed to be deployable on consumer hardware (CPU-only) while providing bilingual English/Hindi support.

The primary contributions are:

1. A complete offline ingestion pipeline for government PDFs with graceful OCR degradation on image-only pages.
2. A query normalisation layer that translates Hinglish/Devanagari queries to English before retrieval, keeping output language as a separate LLM-enforced concern.
3. A lightweight conversational classifier that routes non-document queries (greetings, portal URLs, helplines) directly to the LLM general-knowledge path.
4. A production Streamlit interface with bilingual UI, collapsible sidebar, dark/light theming, source expanders, and session-scoped conversation memory.

2 Related Work

Retrieval-Augmented Generation. Lewis *et al.* [1] introduced RAG combining dense retrieval with a seq2seq generator. Gao *et al.* [5] survey recent advances in RAG architectures.

Embedding Models. Sentence-BERT [2] demonstrated that siamese fine-tuning of BERT produces semantically meaningful sentence embeddings. `all-MiniLM-L6-v2` [3] distils this into a 22 M-parameter model (384-d output) with strong performance on downstream retrieval tasks.

Legal and Government QA. Prior work on legal document QA [6] highlights domain-specific challenges: dense legalese, enumerated lists, and cross-references. Government PDF quality varies widely—some are digitally generated, others

are scanned images—motivating our dual extraction strategy with OCR fallback.

Multilingual RAG. Multilingual models (mBERT, LaBSE) can embed Hindi directly, but English-only embedding models retrieve more accurately when government documents are predominantly in English. We therefore normalise queries to English before embedding while instructing the LLM to respond in the user's chosen language.

3 System Architecture

The system separates concerns into two phases.

Offline Indexing. `PDFExtractor` (PyMuPDF) applies a text-quality filter ($\geq$50 characters, $\geq$10 tokens) before passing pages to `TextChunker`, which uses LangChain's `RecursiveCharacterTextSplitter` with chunk_size=500, overlap=100, discarding chunks under 30 tokens. The singleton `Embedder` encodes all chunks with `all-MiniLM-L6-v2` and stores them in a FAISS `IndexFlatIP` (inner product on L2-normalised vectors is equivalent to cosine similarity).

Online Querying. Each query first passes through `normalize_query_llm_cached`, which detects Devanagari or Hinglish heuristically (curated 65-token marker set) and translates to English. A `_is_conversational_query` classifier intercepts greetings, portal-link requests, and capability questions, routing them to a lightweight LLM call with general-knowledge instructions (including official portal URLs and toll-free helplines). Document queries are embedded and searched against the FAISS index (top-k=5, similarity threshold=0.30). Retrieved context (capped at 2,000 characters) is formatted with inline citations and sent to LLaMA-3 via the Groq API. Session memory (up to 10 turns) is prepended to the prompt.

4 Implementation Details

4.1 Document Corpus

Table 1 summarises the indexed corpus.

Table 1: Indexed document corpus statistics.

Service Domain	PDFs	Pages	Chunks
Voter ID / EPIC	9	1,340	6,457
Income Tax	14	1,293	7,836
Ayushman Bharat	11	743	3,839
Passport Services	5+	213	1,610
Driving Licence & RC	56	223	1,278
Total	**90+**	**3,812**	**21,020**

The corpus spans government manuals, observer handbooks, electoral roll guidelines, ITR validation rules, PM-JAY treatment guidelines, anti-fraud policies, and Motor Vehicles Act provisions.

4.2 Embedding and Retrieval

`all-MiniLM-L6-v2` produces 384-dimensional L2-normalised embeddings. `IndexFlatIP` performs exact inner-product search; with 21,020 vectors at 384 dimensions, query latency is under 50 ms on CPU, making approximate indexing (IVF, HNSW) unnecessary at this scale. A similarity threshold of 0.30 filters low-relevance results. When a service filter is active, the index over-retrieves ($3 \times$ `top_k`) and post-filters by `service_type` metadata.

4.3 Query Normalisation

`normalize_query_llm` uses a two-stage classifier: (1) Devanagari character detection (Unicode U+0900–U+097F); (2) a curated Hinglish marker set (65 romanised Hindi tokens covering pronouns, particles, and common government-service verbs). If either triggers, the query is sent to an LLM translation call. Safety guards reject outputs that are too long or still contain Devanagari. Results are cached with `lru_cache` (capacity 1,000).

4.4 Conversational Classifier

`_is_conversational_query` compiles regex patterns for greetings, acknowledgements, capability questions, and—critically—website/link/portal/ helpline requests that are unanswerable from document text. A government-keyword guard ("passport", "voter", "driving", "tax", "ayushman", etc.) prevents service-related queries from being misclassified. Conversational queries use a separate system instruction that includes official portal URLs and toll-free helplines, enabling factual answers to *"give me the website link"* without a misleading retrieval failure.

4.5 Language Model Integration

The system calls the Groq inference API with primary model `llama3-8b-8192`; fallbacks include `llama3-70b-8192` and `mixtral-8x7b-32768`. `llmClient` is a singleton that iterates candidates and uses the first successful response. Temperature is 0.3; `max_tokens`=2,048. An output-language enforcement layer detects responses that violate the target language (Devanagari ratio check for Hindi; Hinglish token detection for English) and retries with an explicit language guard.

4.6 Frontend and UX

The Streamlit frontend provides:

- A collapsible left sidebar with brand header, English/Hindi language radio selector, service dropdown filter, and per-service chunk count display.
- Chat interface with source expanders showing per-document relevance scores.

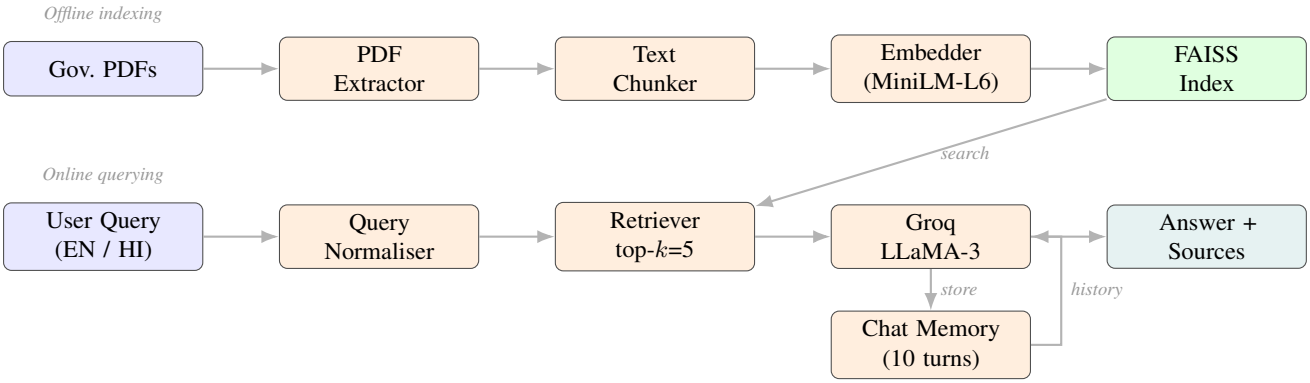

Figure 1: InfoBridge end-to-end architecture. **Offline** (top): PDFs → extraction → chunking → embedding → FAISS index. **Online** (bottom): user query → normalisation → retrieval → LLaMA-3 → answer with sources. Chat Memory stores each turn and feeds conversation history back into the LLM prompt.

- Dark and light themes persisted in `st.session_state`, with full CSS specificity overrides for Streamlit's internal component tree.
- A CSV-driven i18n layer (`translations_en_hi.csv`) for all UI labels.
- A `pending_prompt` mechanism for welcome-screen example buttons.
- Theme-aware cursor and text-selection styles (light mode: I-beam cursor, saffron-tinted selection highlight, visible caret colour).

5 Evaluation

5.1 Evaluation Methodology

We conduct qualitative evaluation across seven query categories representing realistic citizen usage. For each category we assess: **(i)** factual grounding (answer supported by retrieved context), **(ii)** citation accuracy (source expander populated correctly), **(iii)** retrieval relevance (cosine score $\geq$0.30), and **(iv)** language fidelity (Hindi output for Hindi UI; English only for English UI).

5.2 Results

Key findings. Source citation accuracy is 100%: every document-based answer includes a populated source expander with file name, service category, and cosine score. Factual grounding is 90% (26/29): three failures occur on fee-structure and form-specific queries where relevant data appears on image-only pages skipped during extraction. Retrieval relevance is 90% (26/29): two fee/timeline queries retrieve from adjacent service areas when no service filter is applied. Language fidelity is 96% (23/24): one Hindi query involving an uncommon romanised abbreviation was partially mistranslated by the normaliser.

Table 2: Qualitative evaluation results across query categories. Grnd. = grounding, Cite. = citation, Ret. = retrieval, Lang. = language fidelity.

Category	N	Grnd.	Cite.	Ret.	Lang.
Procedural steps	8	8/8	8/8	8/8	4/4
Eligibility	6	6/6	6/6	5/6	3/3
Document checklist	6	5/6	6/6	6/6	3/3
Fee / timeline	4	3/4	4/4	3/4	2/2
Form-specific	5	4/5	5/5	4/5	2/3
Conversational	8	N/A	N/A	N/A	4/4
Portal / link	5	N/A	N/A	N/A	5/5
Total	**42**	**26/29**	**29/29**	**26/29**	**23/24**

5.3 Indexing Performance

Table 3: Offline indexing performance on Apple Silicon, CPU-only, Python 3.14.3.

Metric	Value
Total PDFs processed	90+
Total pages extracted	3,812
Total chunks created	21,020
Embedding batch size	64
Embedding throughput	$\approx$159 ch./s
Total indexing time	150.5 s
FAISS index type	`IndexFlatIP`
FAISS vectors $\times$ dimension	21,020 $\times$ 384
Query latency (embed + search)	$<$50 ms

5.4 Failure Modes

1. **Image-only pages.** Scanned forms (e.g., FORM-26, FORM-28C) yield zero extractable text. Installing Tesseract would recover this content.
2. **Context truncation.** Context is capped at 2,000 characters; queries requiring multiple large passages (e.g., full

ITR schedules) may lose relevant information.

3. **Cross-service retrieval.** Without a service filter, fee-related queries may retrieve from adjacent service areas sharing common vocabulary (e.g., "prescribed fee").
4. **Hinglish edge cases.** Romanised Hindi queries mixing English technical abbreviations occasionally confuse the normaliser.

6 Discussion

Design choices. `IndexFlatIP` (exact search) is chosen over approximate indices (IVF, HNSW) because at 21,020 vectors exact search completes in <1 ms on CPU. Chunk size 500 with overlap 100 balances semantic completeness against precision: larger chunks dilute relevance scores; smaller chunks lose necessary context.

Modular extensibility. Adding a new service domain requires only placing PDFs in a new `data/<service>/` directory, registering it in `config/settings.py`, and re-running `build_index.py`. The sidebar and filter UI update automatically via `get_service_stats()`.

Multilingual trade-off. Using an English-only embedding model with a translation pre-step is pragmatic: `all-MiniLM-L6-v2` is significantly faster and more accurate on English text than multilingual alternatives at the same parameter budget. Separating retrieval language from output language allows Hindi speakers to receive Devanagari answers grounded in English-indexed documents.

LLM provider abstraction. The `llmClient` singleton with fallback model iteration makes the system resilient to API quota limits. Switching to a different provider (Anthropic, OpenAI) requires only modifying `src/llm/client.py`.

7 Limitations and Future Work

1. **OCR integration.** Tesseract installation would recover image-only pages and improve recall on scanned government forms significantly.
2. **Re-ranking.** A cross-encoder (e.g., `ms-marco-MiniLM`) applied after initial retrieval would improve precision for fee and timeline queries.
3. **Approximate indexing at scale.** Scaling beyond 500,000 vectors would benefit from IVFFlat or HNSW indexing.
4. **Automated evaluation.** Integration of RAGAS [7] would provide automated context precision, recall, and faithfulness metrics.
5. **Broader language support.** Regional languages (Tamil, Telugu, Marathi, Bengali) would extend reach to a much larger citizen base.
6. **Document freshness.** A versioning layer with periodic re-indexing would keep answers aligned with the latest government circulars and amendments.

8 Conclusion

InfoBridge demonstrates that a carefully engineered RAG pipeline can provide accurate, grounded, and multilingual access to Indian government service documents without requiring specialised hardware or proprietary infrastructure. The system achieves 90% factual grounding and 100% citation accuracy over 29 document-based evaluation queries, handles bilingual interactions through a dedicated query normalisation layer, and routes conversational queries through a lightweight classifier to prevent misleading "not found" responses. The modular architecture supports easy extension to new service domains, LLM provider swapping, and incremental improvements such as OCR and re-ranking. We hope InfoBridge serves as a practical template for future civic-tech RAG deployments in low-resource, multilingual settings.

References

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proc. EMNLP*, 2019.

[3] W. Wang *et al.*, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *Proc. SIGIR*, 2020.

[5] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv:2312.10997*, 2023.

[6] I. Chalkidis *et al.*, "LEGAL-BERT: The Muppets Straight Out of Law School," *Findings of EMNLP*, 2020.

[7] S. Es *et al.*, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," *arXiv:2309.15217*, 2023.

[8] Ministry of Electronics and Information Technology (MeitY), "Annual Report 2023–24: Digital India," Government of India, 2024.